M-Chain: Decentralized Multi-Party Computation Custody with Quantum-Safe Threshold Signatures

Zach Kelling* Vishnu Seesahai
Lux Industries
research@lux.network

2023

*Corresponding author: zach@lux.network

RESEARCH

Abstract

We present **M-Chain**, a purpose-built blockchain for decentralized multi-party computation (MPC) custody of cross-chain assets. M-Chain eliminates single points of failure in traditional bridge architectures by implementing on-chain threshold signature coordination, autonomous slashing for non-performing signers, and quantum-resistant dual-signature protocols. The system supports multiple threshold schemes (CGG21 for ECDSA, MuSig2 for Bitcoin Taproot, FROST for EdDSA, and Ringtail for post-quantum security) with sub-200ms signature generation latency. M-Chain introduces the **SwapSigTx** transaction type, providing cryptographic proof that a threshold quorum has authorized a cross-chain transfer, replacing centralized swap databases with auditable on-chain logic. Economic incentives align validator behavior through per-signature rewards and time-based slashing penalties. We demonstrate custody of Bitcoin, Ethereum, and XRPL assets with >99.9% uptime and zero security incidents across 10,000+ daily swaps processing $20M+ in volume.

1 Introduction

Cross-chain asset transfers traditionally rely on centralized custodians or trusted committees, creating single points of failure and security vulnerabilities. Recent bridge exploits have resulted in losses exceeding $2B [1], primarily due to compromised private keys, malicious insiders, or centralized infrastructure failures.

1.1 The Bridge Trilemma

Existing bridge architectures face three conflicting requirements:

1. **Security**: No single point of compromise
2. **Performance**: Sub-second finality for user experience
3. **Decentralization**: Permissionless participation

Most bridges sacrifice one or more of these properties:

- **Centralized bridges**: Fast but trusted (e.g., centralized exchanges)
- **Multi-sig bridges**: Decentralized but slow and quantum-vulnerable
- **Light client bridges**: Trustless but expensive and limited cross-chain support

1.2 M-Chain's Solution

M-Chain solves the bridge trilemma through:

1. **Threshold Custody**: t-of-n signatures required, no single point of compromise
2. **On-Chain Coordination**: SwapSigTx provides transparent, auditable proof of authorization
3. **Economic Security**: Staking, rewards, and slashing align validator incentives
4. **Quantum Resistance**: Dual signatures (classical + post-quantum) for future-proofing
5. **Sub-Second Finality**: Optimized MPC protocols achieve ¡200ms signature generation

2 System Architecture

2.1 M-Chain Overview

```
         +--------------------------------------+
         |              M-Chain VM              |
         |--------------------------------------|
         |  \textbullet{} KeyShareRegistry (G1, G2, PK)      |
         |  \textbullet{} SwapSigTx verifier                 |
         |  \textbullet{} SLA / Slashing manager             |
         |  \textbullet{} RewardDistributor                  |
         +------------------+-------------------+
                            |
WarpMsg<MProof>             | gRPC /sign_swap(id)
                            v
+-----------+     +---------------------+     +-----------+
| X-Chain   |<----| mpckeyd (per signer)|---->| BTC / ETH |
| SwapFx    |     +---------------------+     |  XRPL...    |
+-----------+                                  +-----------+
```

Key Components:

- **M-Chain VM**: Coordinates threshold signature generation and validates SwapSigTx
- **mpckeyd**: Validator-side daemon holding threshold key shares
- **X-Chain Integration**: Consumes M-Chain proofs via Warp messaging
- **External Chains**: Bitcoin, Ethereum, XRPL, etc.

2.2 Consensus and Validator Set

Consensus Engine: Lux consensus with 2-second finality
Staking Requirements:

- Minimum stake: 5,000 LUX per MPC signer
- Committee sizes: BTC $\approx$ 15, ETH $\approx$ 15, XRPL $\approx$ 10
- Threshold: $t = \lceil \frac{2n}{3} \rceil$ (e.g., 11-of-15 for BTC)

Economic Alignment:

$$\text{Validator Reward} = \text{Base Staking} + \text{MPC Fees} \tag{1}$$

$$\text{MPC Fee per Swap} = 0.5 \text{ LUX} \times \frac{1}{t} \tag{2}$$

$$\text{Daily Revenue} \approx 10{,}000 \text{ swaps} \times 0.5 \text{ LUX} = 5{,}000 \text{ LUX} \tag{3}$$

3 Threshold Signature Schemes

M-Chain supports multiple threshold signature protocols optimized for different blockchains:

Protocol	Curve	Target Chain	Latency
CGG21	secp256k1	Ethereum, BSC	80ms (15-of-15)
MuSig2	secp256k1	Bitcoin Taproot	45ms (15-of-15)
FROST	Ed25519	XRPL, Solana	35ms (10-of-10)
Ringtail	Lattice (LWE)	Quantum-safe	7ms (15-of-21)

3.1 CGG21: ECDSA Threshold Signatures

CGG21 [2] provides UC-secure threshold ECDSA without trusted dealers.

Setup Phase (Distributed Key Generation):

function DISTRIBUTEDKEYGEN(n, t)
for each participant $i \in [n]$ **do**
    $x_i \leftarrow \text{Random}(Z_q)$ ▷ Secret share
    $X_i \leftarrow x_i \cdot G$ ▷ Public commitment
    Broadcast X_i to all participants
end for
$PK \leftarrow \sum_{i=1}^{n} X_i$ ▷ Aggregate public key
return $(PK, \{x_1, \ldots, x_n\})$
end function

Signing Protocol:

function THRESHOLDSIGN($message, \{x_{ii \in S}, PK$ where $|S| \geq t$
Round 1: Generate ephemeral shares
for each signer $i \in S$ **do**
    $k_i \leftarrow \text{Random}(Z_q)$
    $R_i \leftarrow k_i \cdot G$
    Broadcast R_i
end for

Round 2: Compute partial signatures
$R \leftarrow \sum_{i \in S} R_i$
$r \leftarrow R.x \mod q$ ▷ x-coordinate
$e \leftarrow \text{Hash}(message)$
for each signer $i \in S$ **do**
    $s_i \leftarrow k_i^{-1}(e + r \cdot x_i) \mod q$
    Broadcast s_i
end for

Combine: Aggregate signature
$s \leftarrow \sum_{i \in S} s_i \mod q$
return (r, s) ▷ ECDSA signature
end function

Security Properties:

- UC-secure against malicious adversaries
- t-privacy: $< t$ shares reveal nothing about private key
- Non-interactive with preprocessing

3.2 MuSig2: Bitcoin Taproot Aggregation

MuSig2 [3] provides Schnorr signature aggregation for Bitcoin.

function MuSig2Sign($message, \{sk_{ii \in S}$
KeyGen: Compute aggregate public key
$L \leftarrow \text{Hash}(\{pk_1, \ldots, pk_n\})$
for each i **do**
    $a_i \leftarrow \text{Hash}(L, pk_i)$
end for
$PK \leftarrow \sum_i a_i \cdot pk_i$

Nonce Exchange:
for each i **do**
    $(r_i^1, r_i^2) \leftarrow (\text{Random}(), \text{Random}())$
    $(R_i^1, R_i^2) \leftarrow (r_i^1 \cdot G, r_i^2 \cdot G)$
    Broadcast (R_i^1, R_i^2)
end for

Signing:
$R \leftarrow \sum_i R_i^1 + b \sum_i R_i^2$ ▷ $b = \text{Hash}(PK, R_i^*, m)$
$c \leftarrow \text{Hash}(PK, R, message)$
for each i **do**
    $s_i \leftarrow r_i^1 + b \cdot r_i^2 + c \cdot a_i \cdot sk_i$
    Broadcast s_i
end for
$s \leftarrow \sum_i s_i$
return (R, s) ▷ Schnorr signature
end function

3.3 FROST: Flexible EdDSA Threshold

FROST [4] provides threshold EdDSA for XRPL and Solana.

function FrostSign($message, \{share_{ii \in S}, threshold$
Commitment Phase:
for each $i \in S$ **do**
    $(d_i, e_i) \leftarrow (\text{Random}(), \text{Random}())$
    $(D_i, E_i) \leftarrow (d_i \cdot B, e_i \cdot B)$ ▷ B is EdDSA base
    Broadcast (D_i, E_i)
end for

Signature Generation:
$\rho_i \leftarrow \text{Hash}(i, m, \{D_j, E_j\}_{j \in S})$ for all i
$R \leftarrow \sum_{i \in S}(D_i + \rho_i \cdot E_i)$
$c \leftarrow \text{Hash}(R, PK, message)$
for each $i \in S$ **do**
    $\lambda_i \leftarrow \text{LagrangeCoeff}(i, S)$
    $z_i \leftarrow d_i + \rho_i \cdot e_i + \lambda_i \cdot share_i \cdot c$
    Broadcast z_i

end for
$z \leftarrow \sum_{i \in S} z_i$
return (R, z) ▷ EdDSA signature
end function

3.4 Ringtail: Post-Quantum Threshold

Ringtail [5] provides lattice-based threshold signatures.

function RINGTAILSIGN($message, \{share_{ii \in S}$
Lattice Setup: $n = 1024, q = 2^{32} - 5$

Round 1: Share Generation
for each $i \in S$ **do**
  $y_i \leftarrow \text{SampleGaussian}(\sigma = 3.2)^n$
  $w_i \leftarrow A \cdot y_i \mod q$ ▷ A is public matrix
  Broadcast w_i
end for

Round 2: Challenge Computation
$w \leftarrow \sum_{i \in S} w_i \mod q$
$c \leftarrow \text{Hash}(w, message)$ ▷ Challenge

Round 3: Response Generation
for each $i \in S$ **do**
  $z_i \leftarrow y_i + c \cdot share_i$
  Broadcast z_i
end for

Combine:
$z \leftarrow \sum_{i \in S} z_i$
return (w, z, c) ▷ Lattice signature
end function

Security: Based on LWE (Learning With Errors) hardness, resistant to Shor's algorithm.

4 SwapSigTx: On-Chain Signature Proof

The core innovation of M-Chain is the **SwapSigTx** transaction type, which provides cryptographic proof that a threshold quorum has authorized a swap.

4.1 Transaction Format

struct SwapSigTx:
  $SwapID$: Transaction ID from X-Chain
  $AssetID$: Asset being transferred (BTC, ETH, etc.)
  $MPCAlgo$: Signature algorithm (0=MuSig2, 1=CGG21, 2=FROST)
  $Signature$: Threshold signature bytes
  $SigBitmap$: Bitmap of participating signers
  $ProofHash$: Hash of signing transcripts (audit trail)

4.2 Validation Rules

```
function ValidateSwapSigTx(tx, state)
Check 1: Retrieve aggregate public key
PK ← state.KeyRegistry[tx.AssetID]

Check 2: Verify threshold met
signerCount ← PopCount(tx.SigBitmap)
threshold ← state.Threshold[tx.AssetID]
if signerCount < threshold then
    return INVALID_THRESHOLD
end if

Check 3: Verify signature
msgHash ← Hash(tx.SwapID)
valid ← AggVerify(PK, tx.SigBitmap, tx.Signature, msgHash)
if not valid then
    return INVALID_SIGNATURE
end if

Check 4: Prevent double-signing
if state.SwapState[tx.SwapID] = SIGNED then
    return ALREADY_SIGNED
end if

return VALID
end function
```

4.3 State Transition

Upon successful validation:

1. Credit `rewardPerSig` to each signer in bitmap
2. Mark `swapState`$[SwapID] =$ `SIGNED`
3. Generate Warp message proof for X-Chain
4. Emit `SwapSigned` event

5 Dual-Signature Quantum Security

M-Chain implements a phased approach to quantum resistance through dual signatures.

5.1 DualSigTx Transaction Type

```
struct DualSigTx:
    SwapID: X-Chain transaction ID
    AssetID: Asset identifier
    ClassicalSig: CGG21/MuSig2 signature
```

```
    ClassicalBitmap: Classical signers
    QuantumSig: Ringtail signature
    QuantumBitmap: Quantum signers
    ProofHash: Combined proof hash
```

5.2 Phase Transition

Phase	Classical Sig	Quantum Sig	Validity
Phase 0	Required	Not generated	Classical only
Phase 1	Required	Generated	Classical validates
Phase 2	Required	Required	Both validate
Phase 3	Optional	Required	Quantum primary

```
function ValidateDualSig(tx, state)
phase ← state.QuantumPhase

if phase ≥ 1 then
    Verify classical signature
    validClassical ← VerifyCGG21(tx.ClassicalSig, ...)
    if not validClassical then
        return INVALID_CLASSICAL
    end if
end if

if phase ≥ 2 then
    Verify quantum signature
    validQuantum ← VerifyRingtail(tx.QuantumSig, ...)
    if not validQuantum then
        return INVALID_QUANTUM
    end if
end if

return VALID
end function
```

5.3 Security Analysis

Adversary Type	Classical Sig	Result
Classical attacker	Secure (128-bit)	Safe
Quantum attacker (Phase 0)	Vulnerable	Vulnerable
Quantum attacker (Phase 1)	Vulnerable	Safe (redundancy)
Quantum attacker (Phase 2+)	Vulnerable	Safe (required)
Both compromised	Compromised	Unsafe

6 Economic Model and Incentives

6.1 Reward Structure

Per-Signature Rewards:

$$\text{Reward per signer} = \frac{\text{rewardPerSig}}{t} \tag{4}$$

$$= \frac{0.5 \text{ LUX}}{15} \approx 0.033 \text{ LUX per swap} \tag{5}$$

Daily Revenue (10,000 swaps):

$$\text{Daily earnings} = 10{,}000 \times 0.033 \text{ LUX} \tag{6}$$

$$\approx 333 \text{ LUX per signer} \tag{7}$$

$$\approx \$10{,}000 \text{ per month at } \$1 \text{ LUX} \tag{8}$$

6.2 Slashing Conditions

Violation	Evidence	Penalty
Missed deadline	Swap expired unsigned	20% stake
Double signing	Two conflicting sigs	100% stake
Invalid signature	Verification fails	75% stake
Extended downtime	99%+ missed swaps	25% stake

function CheckSlashing($swapID, state$)
$swap \leftarrow state.Swaps[swapID]$
if CurrentTime() $> swap.Deadline +$ GraceBlocks **then**
  if $swap.State =$ `PENDING` **then**
    $signers \leftarrow state.ActiveSigners[swap.Asset]$
    for each $signer \in signers$ **do**
      $penalty \leftarrow signer.Stake \times 0.20$
      $signer.Stake \leftarrow signer.Stake - penalty$
      Burn($penalty \times 0.50$)
      Reward($reporter, penalty \times 0.50$)
      Emit `SignerSlashed`($signer, swapID, penalty$)
    end for
  end if
end if
end function

6.3 Fee Distribution

7 Cross-Chain Swap Lifecycle

7.1 Complete Flow

Step 1: User submits $SwapTx$ on X-Chain

Recipient	**Percentage**
MPC Signers	60%
DAO Treasury	40%
Slashing Penalties:	
Burned	50%
Reporter Reward	50%

  $SwapTx(src : \text{LUX}, dst : \text{BTC}, amount, recipient)$

Step 2: X-Chain emits `SwapRequested` event

Step 3: M-Chain validators detect event via watcher
  WatchXChain() $\rightarrow$ EnqueueSwap($swapID$)

Step 4: Validators generate threshold signature
  Each *mpckeyd* daemon:
    $share_i \leftarrow$ GenerateShare($swapID$)
    Broadcast($share_i$)

Step 5: Leader aggregates and submits $SwapSigTx$
  Aggregate($\{share_i\}_{i \in S}$) $\rightarrow$ $signature$
  SubmitSwapSigTx($swapID, signature, bitmap$)

Step 6: M-Chain validates and finalizes
  ValidateSwapSigTx() $\rightarrow$ `SUCCESS`
  RewardSigners($\{i \in bitmap\}$)

Step 7: Generate Warp proof for X-Chain
  $proof \leftarrow$ GenerateMProof($swapID, signature$)
  SendWarpMsg($X\text{-}Chain, proof$)

Step 8: X-Chain verifies and settles
  VerifyMProof($proof$) $\rightarrow$ `VALID`
  ExecuteSwap($swapID$) ▷ Unlock/mint assets

Step 9: Broadcast to external chain
  BroadcastTx(BTC, $recipient, amount, signature$)

Step 10: Confirm external finality
  WaitForConfirmations(BTC, $txID, confirms = 6$)

7.2 Failure Recovery

Timeout Handling:

- If swap not signed within deadline $\rightarrow automaticslashingX-Chainrefundsuseraftergraceperiod$

- Slashed funds partially burned, partially rewarded to reporter

External Chain Reorg:

- Monitor external chain for reorgs up to finality depth

- If reorg detected before finality $\rightarrow retrybroadcastIfreorgafterfinality \rightarrow proof-of-non-inclusionrefund$

8 Performance Evaluation

8.1 Latency Breakdown

Operation	**Time**	**Description**
Event detection	50ms	X-Chain watcher polling
MPC coordination	80ms	CGG21 signature (15-of-15)
M-Chain finality	2s	Lux consensus
Warp proof gen	20ms	Merkle proof generation
X-Chain validation	50ms	Proof verification
External broadcast	500ms	Bitcoin/Ethereum tx
Total	**2.7s**	End-to-end swap latency

8.2 Throughput Analysis

M-Chain Block Capacity:

$$\text{SwapSigTx per block} \approx 1{,}000 \tag{9}$$

$$\text{Block time} = 2\text{s} \tag{10}$$

$$\text{Theoretical TPS} = \frac{1{,}000}{2} = 500 \text{ swaps/second} \tag{11}$$

Practical Limits:

- MPC latency (80ms) allows 12 concurrent signing sessions

- Validator bandwidth 10MB/s $\rightarrow$ 5,000$swaps/sCurrentmainnet:$ 10,000$swaps/day($ 0.1$TPS)$

- Headroom: 5,000$\times currentvolume$

8.3 Security Metrics

Mainnet Statistics (6 months):

9 Security Considerations

9.1 Threat Model

Adversary Capabilities:

Metric	Value
Total swaps	1.8M
Total volume	$3.2B
Average daily swaps	10,000
Security incidents	0
Slashing events	3 (false positives)
Validator uptime	99.94%
Signature success rate	99.98%

- Can corrupt up to $f < n/3$ validators (Byzantine)
- Can delay network messages by up to Δ_{max}
- Cannot break cryptographic assumptions (discrete log, lattice hardness)

9.2 Byzantine Fault Tolerance

[Threshold Security] With $t = \lceil \frac{2n}{3} \rceil$ and $f < \frac{n}{3}$ Byzantine validators, an adversary cannot forge a valid threshold signature.

A valid signature requires t shares. With $f < n/3$ Byzantine nodes:

$$\text{Honest shares available} = n - f > n - \frac{n}{3} = \frac{2n}{3} \tag{12}$$

$$\text{Required threshold} = t = \lceil \frac{2n}{3} \rceil \tag{13}$$

Thus, $n - f > t$, ensuring honest validators always have enough shares. Byzantine nodes alone cannot reach threshold t.

9.3 Quantum Security Timeline

Phase	Timeline
Phase 0 (Classical)	2024-2027
Phase 1 (Transition)	2027-2030
Phase 2 (Dual-sig required)	2030-2035
Phase 3 (Quantum primary)	2035+

10 Implementation

10.1 mpckeyd Daemon

function MpckeyMain()
$shares \leftarrow$ LoadKeyShares()
$xchainWatcher \leftarrow$ StartWatcher()

while true **do**

```
    swaps ← xchainWatcher.GetPendingSwaps()
    for each swap ∈ swaps do
        assetType ← swap.GetAssetType()
        share ← shares[assetType]

        if assetType = BTC then
            sig ← MuSig2Sign(swap, share)
        else if assetType = ETH then
            sig ← CGG21Sign(swap, share)
        else if assetType = XRPL then
            sig ← FrostSign(swap, share)
        end if

        BroadcastShare(sig)

        if IsLeader() and HasQuorum() then
            aggSig ← AggregateShares()
            SubmitSwapSigTx(swap.ID, aggSig)
        end if
    end for
    Sleep(100ms)
end while
end function
```

10.2 Key Management

Distributed Key Generation:

- Performed during validator onboarding
- No trusted dealer (all keys generated collaboratively)
- Shares stored in encrypted HSM or TEE
- Regular key rotation (every 90 days)

Share Storage:

- AES-256 encryption at rest
- Hardware security module (HSM) support
- Trusted execution environment (TEE) integration
- Multi-layer access control

11 Future Work

11.1 Proactive Secret Sharing

Implement PSS (Proactive Secret Sharing) for automatic key rotation without coordinator:

- Periodic re-sharing of key shares
- Maintains same public key
- Mitigates slow key leakage attacks
- Target: 30-day rotation cycles

11.2 Hardware Acceleration

- FPGA/ASIC for MPC operations
- Target: $10\times speedup(8mssignatures)GPUaccelerationforRingtaillatticeoperations$

- Hardware security modules for key protection

11.3 Cross-Rollup Support

Extend M-Chain to support L2 rollups:

- Optimism, Arbitrum, zkSync custody
- Fast finality with optimistic execution
- Fraud proof integration
- Native rollup interoperability

12 Concrete Security Analysis (2025 Revision)

12.1 Forgery Probability

We compute concrete security margins for the deployed MPC configurations. For a (t, n)-threshold scheme, the forgery probability depends on the underlying hardness assumption:

Scheme	**Configuration**	**Hardness**	Pr[**Forge**]
CGG21	11-of-15 (ETH)	ECDLP + Paillier	$< 2^{-127}$
MuSig2	11-of-15 (BTC)	OMDL in ROM	$< 2^{-128}$
FROST	7-of-10 (XRPL)	OMDL in ROM	$< 2^{-128}$
Ringtail	15-of-21 (PQ)	LWE ($n = 630$, $q = 2^{32}$)	$< 2^{-128}$
Dual-sig	CGG21 + Ringtail	ECDLP $\times$ LWE	$< 2^{-255}$

12.2 Liveness at Scale

For $n = 100$ validators, $t = 67$ threshold, individual uptime $p = 0.999$:

$$\Pr[\text{unavailable}] = \Pr[\text{Binomial}(100, 0.999) < 67] < 2^{-139.6} \tag{14}$$

This is computed via the exact binomial CDF. The expected available nodes $\mu = 99.9$ exceeds the threshold by 32.9 nodes, providing substantial headroom. See Appendix of the companion Lux Bridge paper [6] for the full derivation.

12.3 Contract-Level Security Fixes

Four critical vulnerabilities were identified in the bridge smart contracts that interact with M-Chain:

1. **C-01**: `bridgeMint()` in LRC20B lacked daily limits. Fixed: configurable `dailyMintLimit` with automatic period reset.
2. **C-02**: `MINTER_ROLE` was conflated with `DEFAULT_ADMIN_ROLE`. Fixed: separate role with explicit grant/revoke.
3. **C-03**: MPC-signed messages used `block.timestamp` as nonce. Fixed: monotonic counter nonce per bridge contract.
4. **C-04**: Withdraw nonce used predictable hash. Fixed: sequential per-user counter nonce.

All fixes verified by Halmos symbolic proofs (68 `check_` functions across the ecosystem).

12.4 Formal Verification

The M-Chain threshold cryptography is backed by Lean 4 proofs at `github.com/luxfi/formal`:

- `Crypto/FROST.lean`: FROST correctness and threshold unforgeability
- `Crypto/BLS.lean`: BLS aggregation soundness from bilinearity
- `Crypto/Ringtail.lean`: LWE security with concrete parameters ($n = 630$, $q = 2^{32}$, $\sigma = 13744$)
- `Consensus/BFT.lean`: $2f + 1$ quorum safety for SwapSigTx validation

13 Conclusion

M-Chain eliminates single points of failure in cross-chain bridges through decentralized threshold custody, on-chain coordination via SwapSigTx, and economic alignment through staking and slashing. The system achieves:

1. **Zero-trust custody**: No single validator can compromise funds
2. **Sub-second signing**: 80ms threshold signature generation
3. **Quantum resistance**: Dual-signature upgrade path to post-quantum security
4. **Economic security**: $100M+ staked with automated slashing
5. **Transparency**: All swap operations auditable on-chain

By replacing centralized swap databases and key managers with decentralized blockchain infrastructure, M-Chain provides the foundation for secure, high-performance cross-chain asset transfers. The phased quantum security approach ensures long-term viability as quantum computing advances.

M-Chain demonstrates that decentralization, security, and performance are not mutually exclusive—threshold cryptography and economic incentives can achieve all three simultaneously.

References

[1] Chainalysis (2022). *Cross-Chain Bridge Hacks Emerge as Top Security Risk*. Chainalysis Blog.

[2] Canetti, R., Gennaro, R., Goldfeder, S., Makriyannis, N., & Peled, U. (2021). *UC Non-Interactive, Proactive, Threshold ECDSA with Identifiable Aborts*. ACM CCS 2021.

[3] Nick, J., Ruffing, T., & Seurin, Y. (2021). *MuSig2: Simple Two-Round Schnorr Multi-Signatures*. CRYPTO 2021.

[4] Komlo, C., & Goldberg, I. (2020). *FROST: Flexible Round-Optimized Schnorr Threshold Signatures*. SAC 2020.

[5] NTT Research (2024). *Ringtail: World's First Two-Round Post-Quantum Threshold Signature Scheme*. Cryptology ePrint Archive.

[6] Kelling, Z. & Bin, W. (2025). *Lux Bridge: Threshold-Secured Cross-Chain Communication with MPC Custody, Post-Quantum Verification, and Formally Verified Contract Invariants*. Lux Network.

[7] Shamir, A. (1979). *How to Share a Secret*. Communications of the ACM, 22(11), 612-613.

[8] Gennaro, R., Jarecki, S., Krawczyk, H., & Rabin, T. (1999). *Secure Distributed Key Generation for Discrete-Log Based Cryptosystems*. EUROCRYPT 1999.

[9] Desmedt, Y., & Frankel, Y. (1990). *Threshold Cryptosystems*. CRYPTO 1989.